SW Test Engineering

Concepts, types and levels

M.Sc. Ruben Adrian Gil Rivera

Before start, a moment of relax

… do you see yourself on this situation?

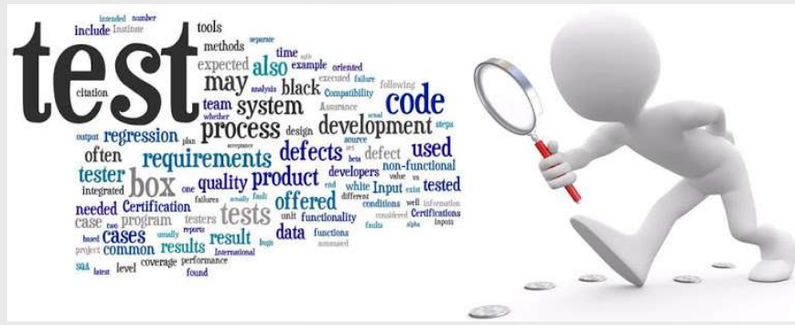


Copyright © 2001 United Feature Syndicate, Inc.

What are your thoughts about It?

Agenda

1. **Introduction & Concepts**
    - Why SW testing?
    - Cost of errors
    - Shades of an error
    - SW Validation
    - SW Verification
2. **Types and Testing levels**
    - Functional Testing (techniques)
    - Non Functional Testing (techniques)
    - SW Testing Levels
        - Module Testing
        - Integration Testing
        - SW System Testing
    - Acceptance Testing
    - Conformance Testing



Software Testing

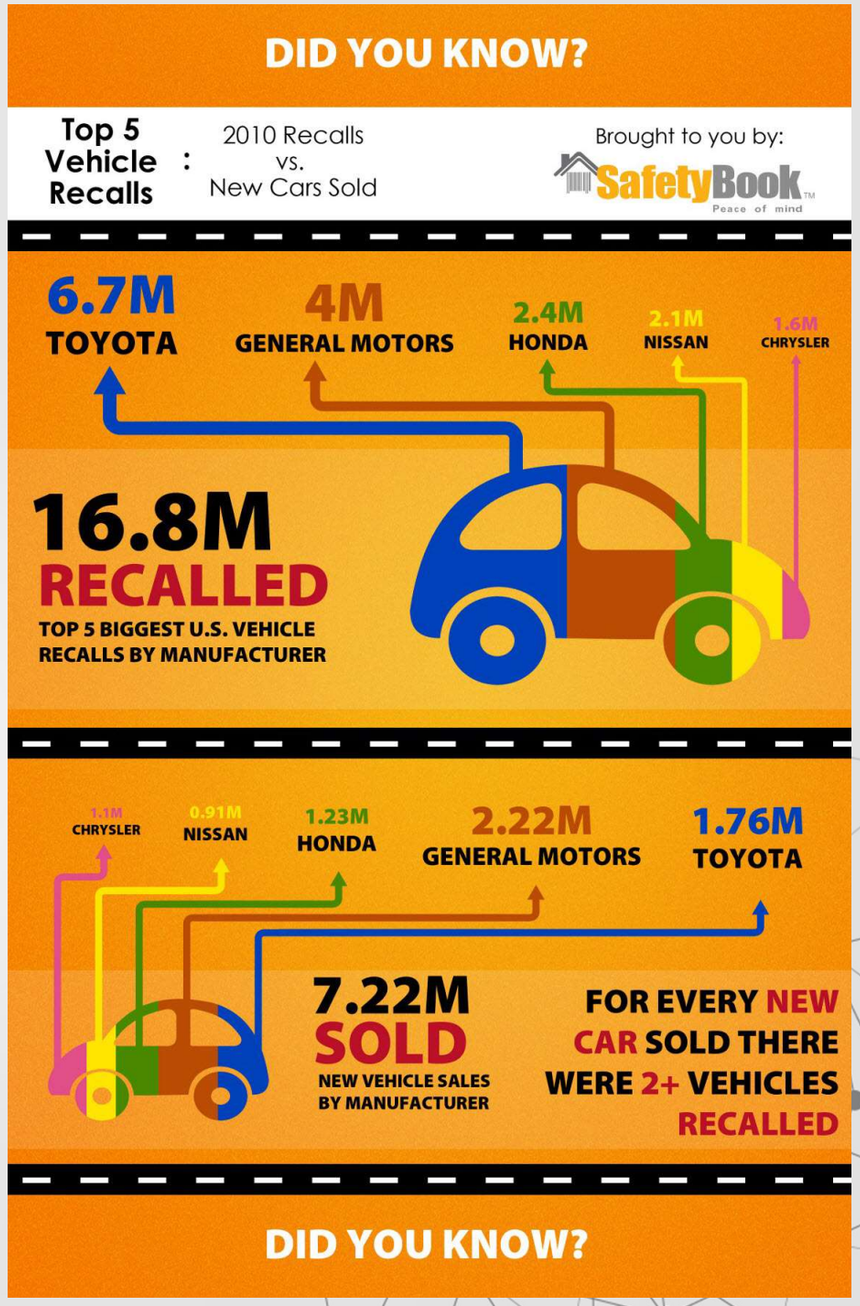
Introduction & Concepts



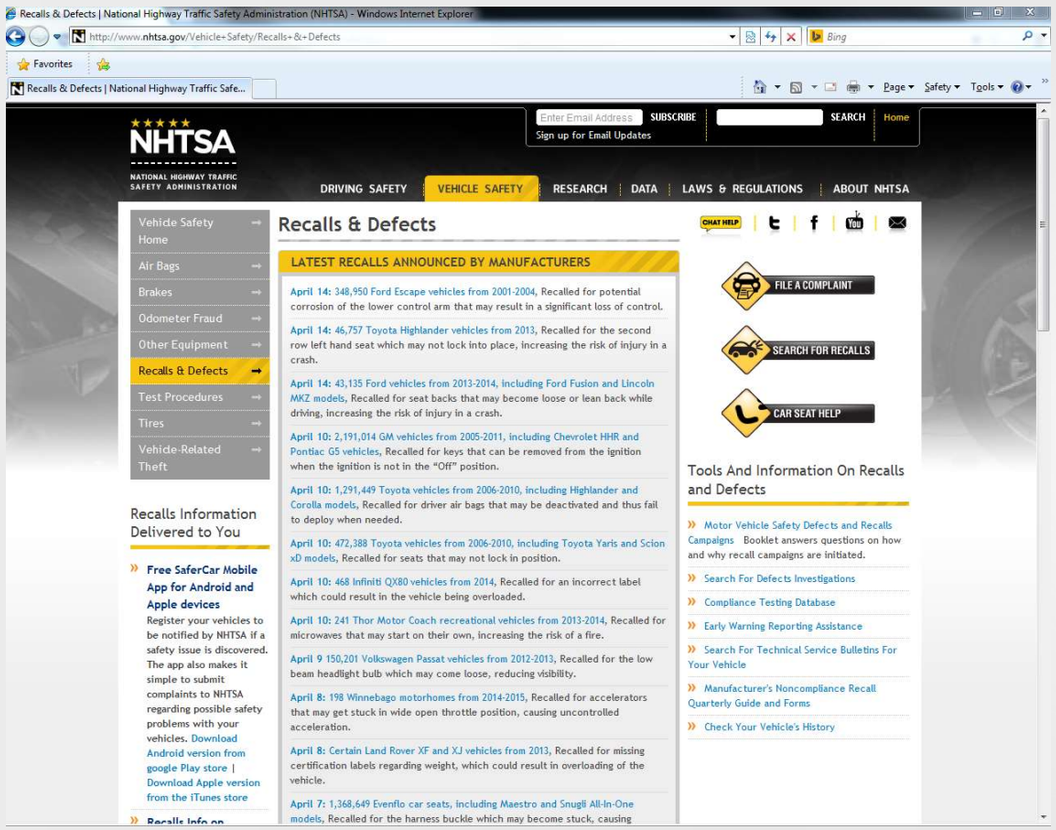
Why SW Testing?

Objectives

- High quality customer requirements
- Recalls are expensive
- Critical security systems
- Very short development cycle



Why SW Testing? - Recalls



Recalls & Defects

LATEST RECALLS ANNOUNCED BY MANUFACTURERS

April 14: 348,950 Ford Escape vehicles from 2001-2004, Recalled for potential corrosion of the lower control arm that may result in a significant loss of control.

April 14: 46,757 Toyota Highlander vehicles from 2013, Recalled for the second row left hand seat which may not lock into place, increasing the risk of injury in a crash.

April 14: 43,135 Ford vehicles from 2013-2014, including Ford Fusion and Lincoln MKZ models, Recalled for seat backs that may become loose or lean back while driving, increasing the risk of injury in a crash.

April 10: 2,191,014 GM vehicles from 2005-2011, including Chevrolet HHR and Pontiac G5 vehicles, Recalled for keys that can be removed from the ignition when the ignition is not in the "Off" position.

April 10: 1,291,449 Toyota vehicles from 2006-2010, including Highlander and Corolla models, Recalled for driver air bags that may be deactivated and thus fail to deploy when needed.

April 10: 472,388 Toyota vehicles from 2006-2010, including Toyota Yaris and Scion xD models, Recalled for seats that may not lock in position.

April 10: 468 Infiniti QX80 vehicles from 2014, Recalled for an incorrect label which could result in the vehicle being overloaded.

April 10: 241 Thor Motor Coach recreational vehicles from 2013-2014, Recalled for microwaves that may start on their own, increasing the risk of a fire.

April 9 150,201 Volkswagen Passat vehicles from 2012-2013, Recalled for the low beam headlight bulb which may come loose, reducing visibility.

April 8: 198 Winnebago motorhomes from 2014-2015, Recalled for accelerators that may get stuck in wide open throttle position, causing uncontrolled acceleration.

April 8: Certain Land Rover XF and XJ vehicles from 2013, Recalled for missing certification labels regarding weight, which could result in overloading of the vehicle.

April 7: 1,368,649 Evenflo car seats, including Maestro and Snugli All-In-One models, Recalled for the harness buckle which may become stuck, causing difficulty removing the child and increasing the risk of injury in the event of an emergency.

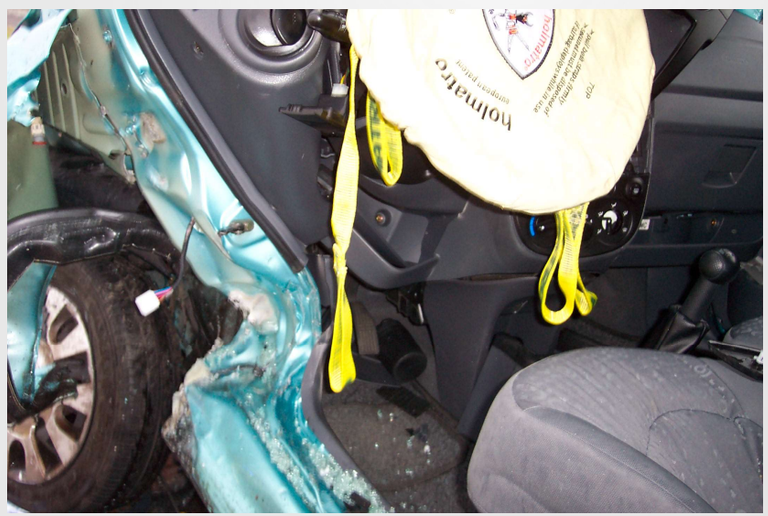
April 7: 708 Keystone RV trailers and fifth wheels from 2014, Recalled for inner wheel bearings that may become overheated and cause the wheel to seize or separate from the vehicle.

April 7: 94 Zero motorcycles from 2014, including Zero DS and Zero SR models

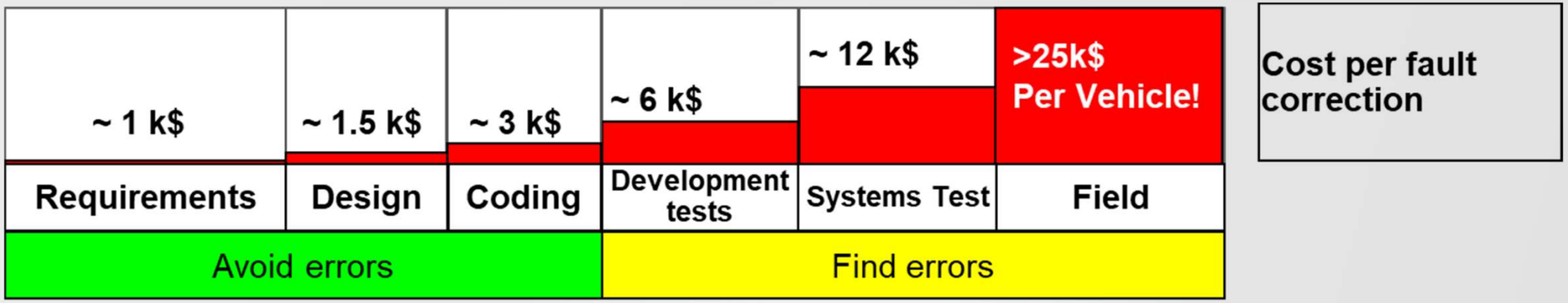
Why SW Testing? - Recalls







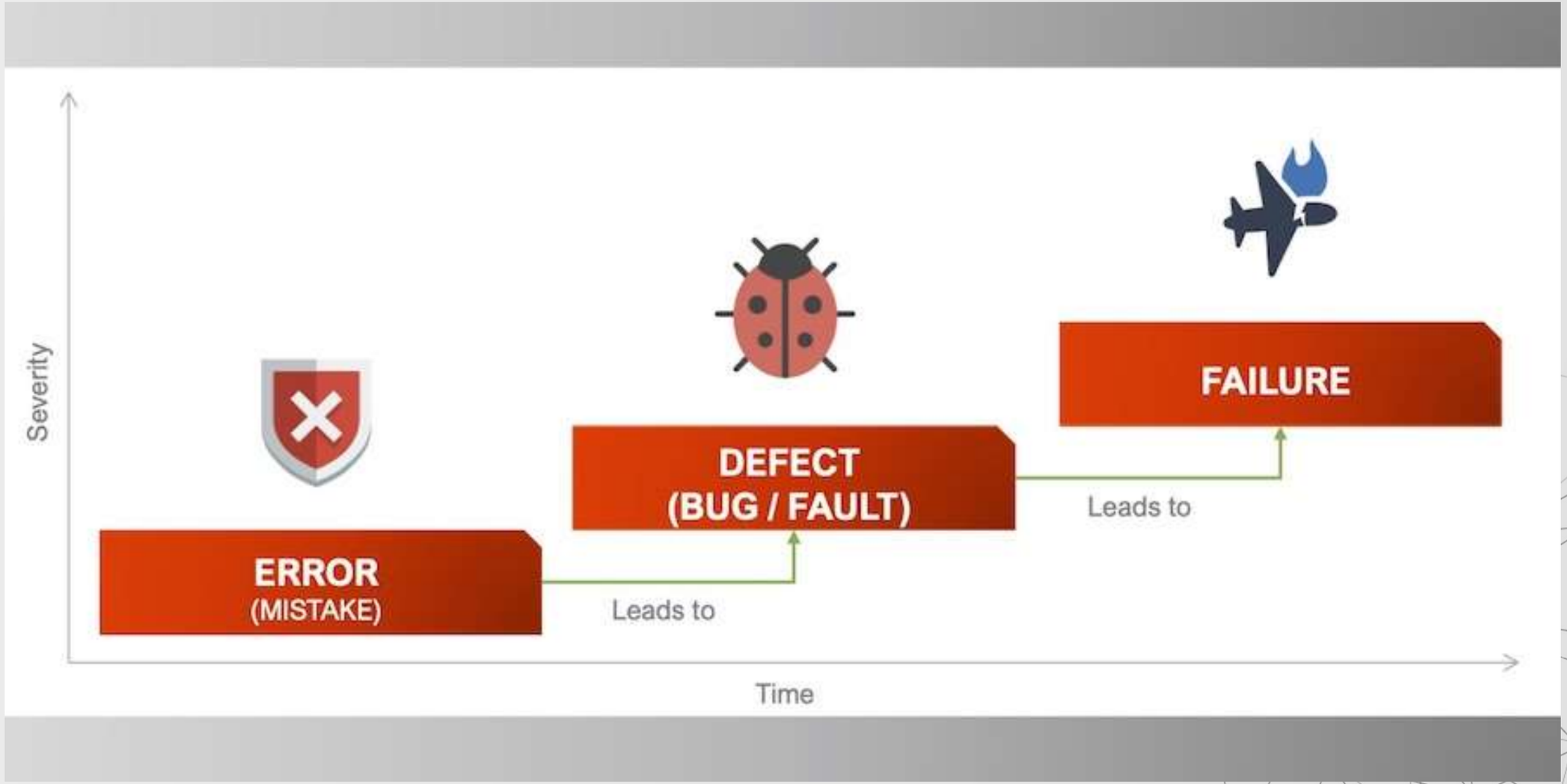
Cost of errors



To reduce the cost of errors, there are 3 paths identified:

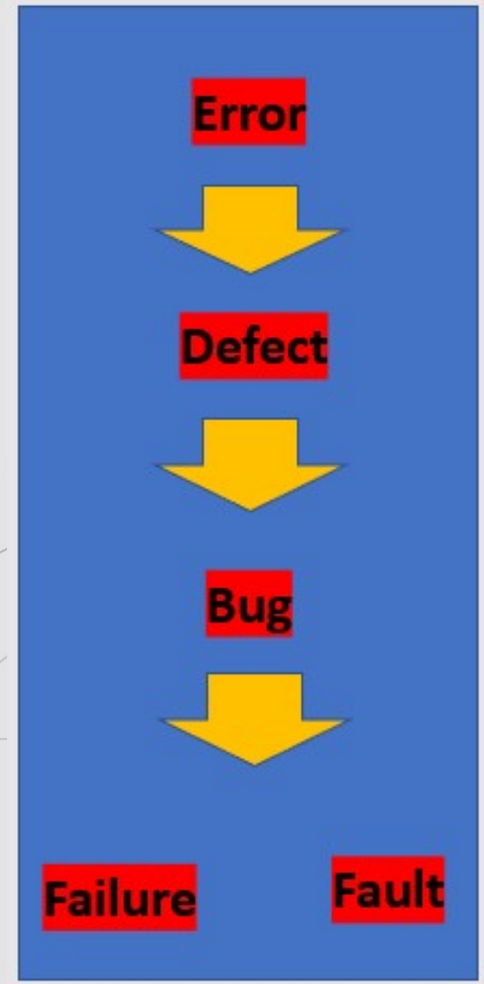
- **Avoid errors** in **the early stages** to decrease the total number of errors
- Corrections **on time** are **cheaper**
- The **sooner an error is detected**, the **less likely** it is that its correction will introduce **new errors**.

Shades of an Error



Shades of an Error – Terminologies

- **Error**: A **mistake in coding** is called Error.
- **Defect**: Error **found by tester** is called Defect. The variation between the actual results and the expected.
    - **Bug/Anomaly**: Defect **accepted by development team**, then is called Bug/Anomaly.
- **Failure**: When a defect **reaches the end customer** it is called a Failure.
- **Missing and Wrong**: A **requirement** of the customer that was **not fulfilled.**
- **Fault**: A fault makes an **application behave in a wrong manner**.



V&V
Verification & Validation

Under SW project management, SW testing and SW engineering:

- Verification and Validation (V&V) are independent processes that are **used together to check that a product, service, or system conforms to requirements and specifications as well as fulfills its intended purpose**.





V&V
Verification & Validation

We can say that we are answering two separate questions:



- Are we doing what we say we do? (Verification)

and

- What we are doing, Is It the right thing? (Validation)



V&V
Verification & Validation - Examples

- **Verification**: An operator checks the graph every 10 minutes, that it is within the temperature margins, and leaves his signature on the graph as **evidence of said review**. Checking all these records is the action of "verifying", checking that every just 10 minutes there is a data.

- **Validation**: This is what **confirms** that what we are doing is being done well and the **intended objective is achieved.**





Software Validation

Inputs/needs

- SW related system info
- Software Requirements Specification SRS
- Software Validation Plan
- Embedded Software

Outcome

- Deliverable Software
- Documentation released
- Validation Report

Activities

- Validate the performance and behavior of the SW against the SW Requirements Specification (SRS).
- Validate that the general behavior meets all the established requirements



Software Verification

Inputs/needs

- Software Verification Plan
- Test Specification from design
- Test Cases (identified in the design phase)

Outcome

- Test Cases
- Verified/reviewed modules
- Verification Report

Activities

- Create/Refine test cases from the Test Specification
- Run specified tests, manual/automatic
- Verification through code review



Software Testing

Types (techniques) and Testing levels



Software Testing

- "It is the process of **analyzing SW elements to detect differences between existing and required conditions** (bugs) and thus evaluate the characteristics of SW elements" [IEEE 829].

- It **consists** of planning, design, implementation, execution, maintenance, test reuse, test documentation and test environment.

- It is a formal activity that aims to **find errors and demonstrate quality** but does not prove the absence of errors nor engages in locating or removing errors.



Software Testing – Testing types

Widely, testing is classified in two types: **Functional** and **Non functional**.

- **Functional testing** consists of testing the 'functionality' of a piece of software or an application under test.
- This type of tests, **verifies the behavior of the software** under test depending on the customer's requirements, a document called a **software specification or Requirements Specification** is used as a guide to test the application.



- **Non functional testing**, checks the **quality** of the software to be tested.
- The quality mainly depends on the time, accuracy, stability, correctness and durability of a product under various adverse circumstances.



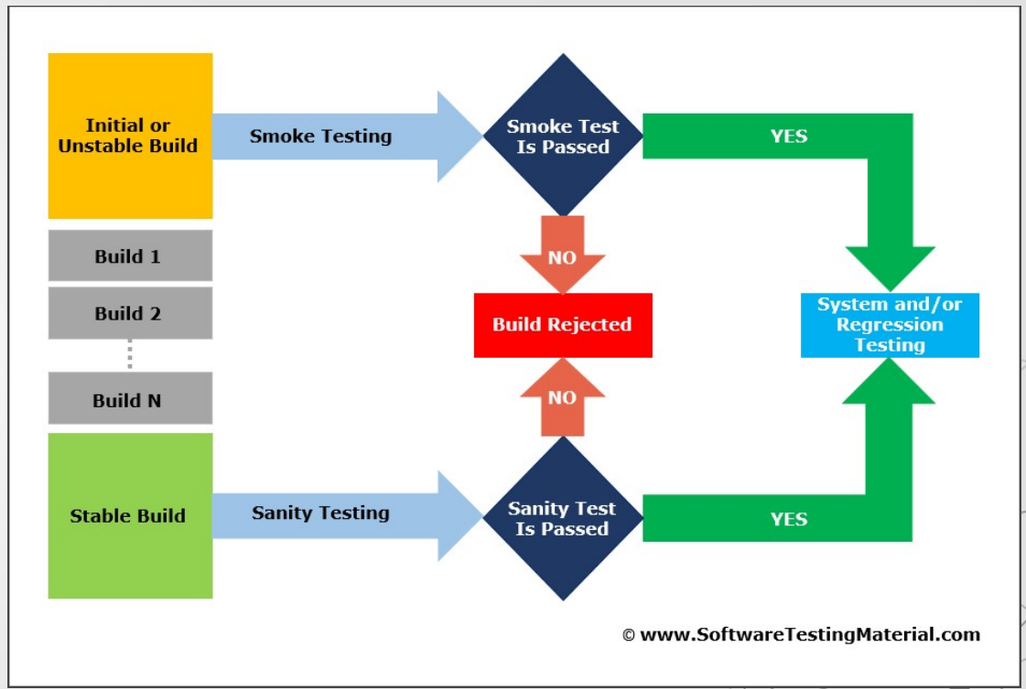
Software Testing – Functional testing (techniques)

- **Smoke test:**

  This type of testing is **done before the actual system** testing **to check if critical functionalities are working well** for further extensive testing. This, in turn, saves time installing the new build again and avoids further testing if critical functionality doesn't work. It is a generalized way of testing the application.

- **Sanity Testing:**

  It is a type of testing where **only a specific functionality is tested or a bug is fixed to check if the functionality works well** and see if there are no other issues due to changes in related components. It is a specific way to test the application.



Software Testing – Functional testing (techniques)

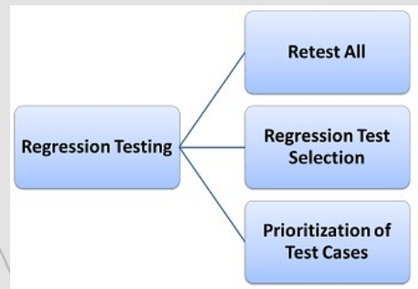
- **Integration testing:**

  It is done when **two or more software functions or components are integrated** to form a system. Basically, it verifies the correct operation of the software when the **components are merged** to work as a single unit.



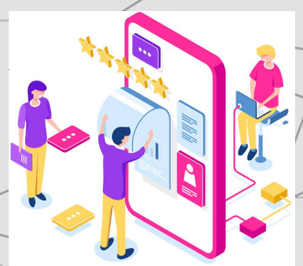
- **Regression Testing:**

  it takes place **upon receipt of the software build after fixing bugs** that **were found in the initial round** of testing. Check if the bugs are fixed and check if all the software is working fine with the changes.



- **User Acceptance Testing:**

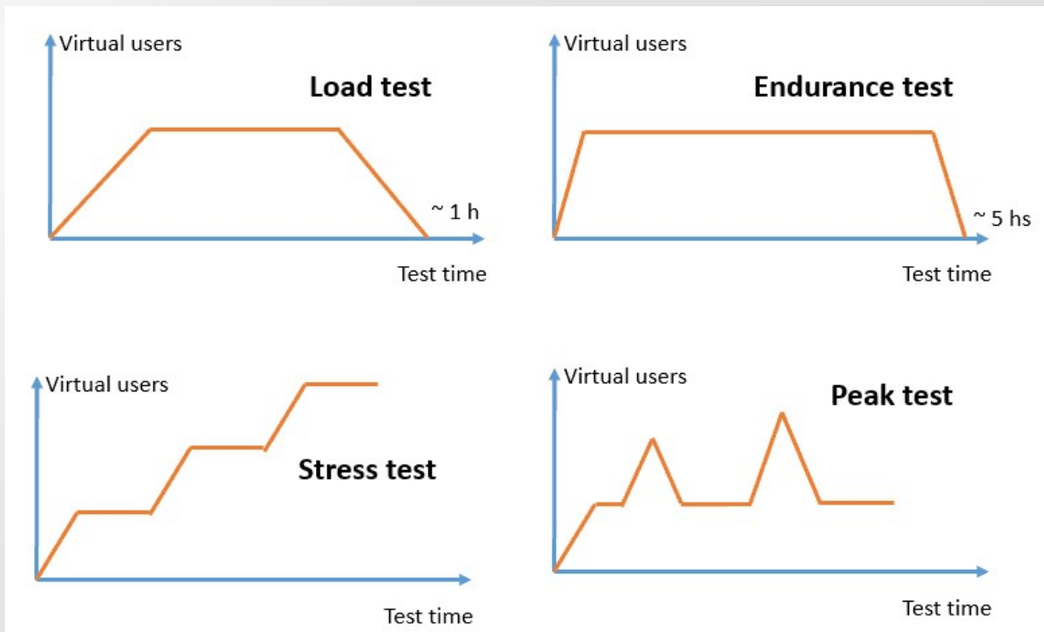
  Actual **end users or customers receive a trial version for use** in their office setup to test if the software performs to their requirements in a real environment. This testing is done before the final release and is also called Beta testing or End User testing.



Software Testing – Non Functional testing (techniques)

- **Performance testing:**
  - **Load Testing** - An application is tested for its response time in a real environment
  - **Stress Testing** – An application is stressed with additional workload to check if it works efficiently and is able to handle the stress as per the requirement.
  - **Resistance Testing** – Checks the endurance power of the software when loaded with a constant workload.





Software Testing – Non Functional testing (techniques)

- **Usability testing:**

  In this type of testing, the user interface is tested for usability and to see how easy it is to use.

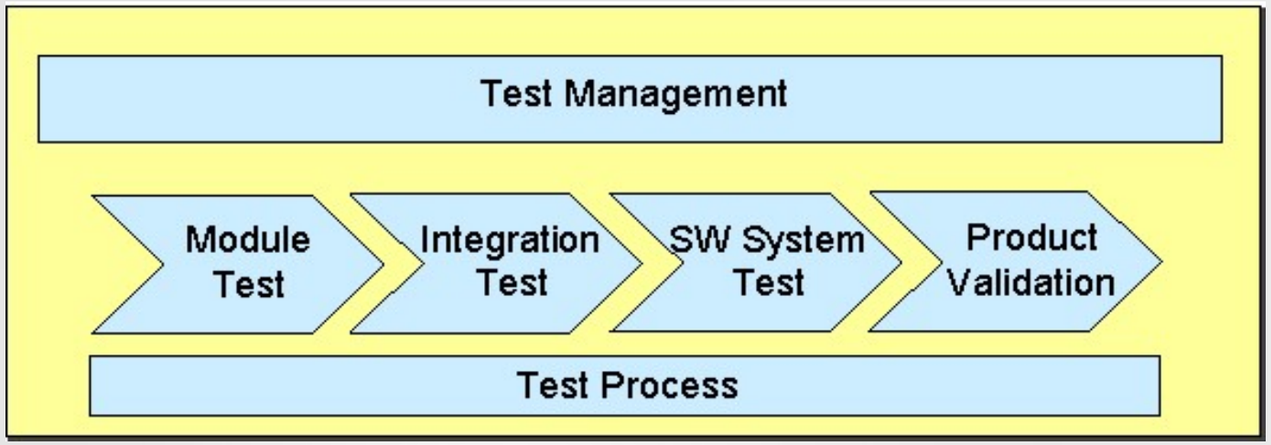


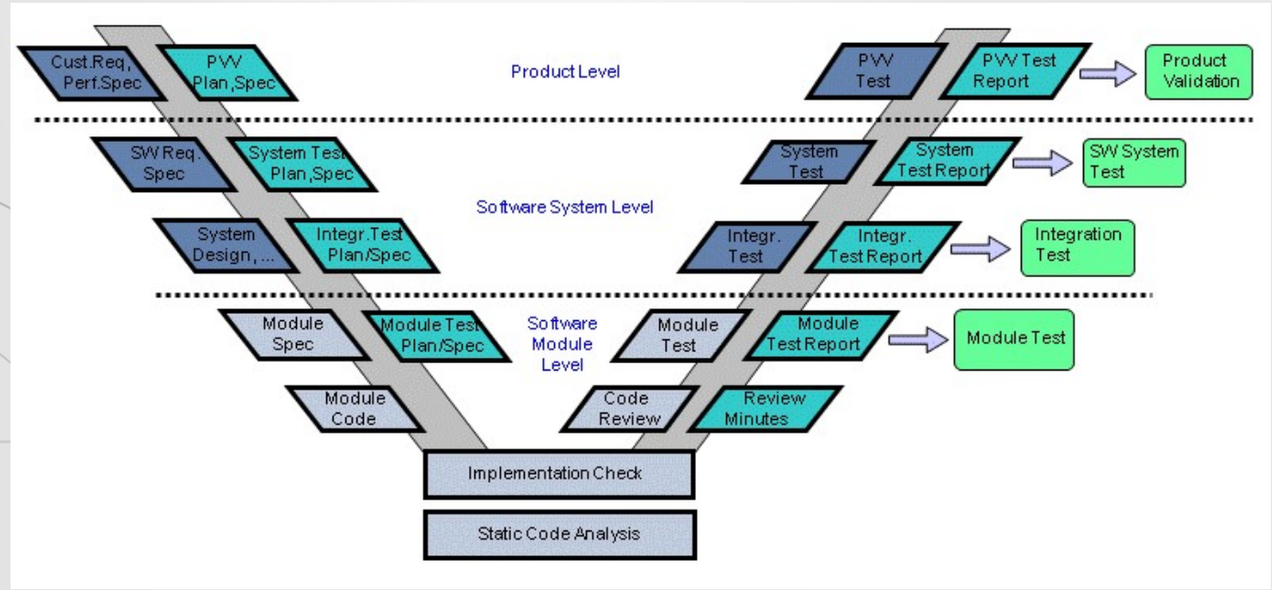
- **Security Testing:**

  Checks how secure the software is regarding the data on the network against malicious attacks. Key areas that will be tested in this test include authorization, authentication of users, and their access to data based on roles such as administrator, moderator, writer, and user level.



Software Testing – SW Testing levels





Software Testing

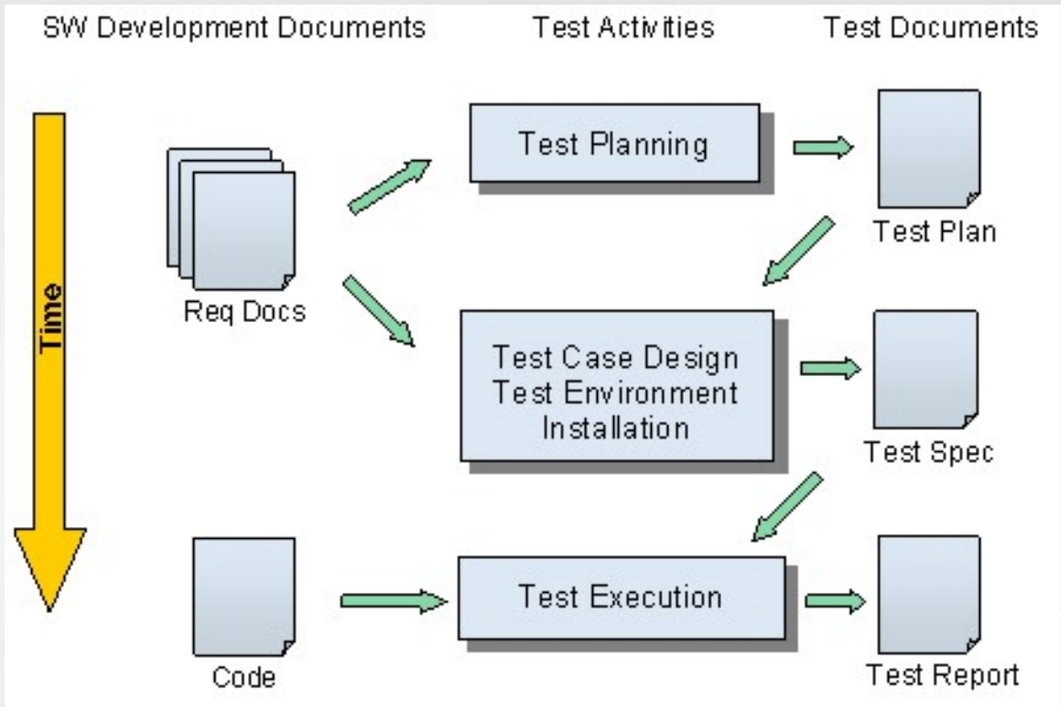
- Each level of SW testing has a particular approach based on its own criteria, goals and techniques, thus ensuring the quality of the element under test for the next level of testing.
- **Errors not detected at one level** of testing **can be detected at the next level**, which is why each level of testing must be **thoroughly documented**.
  - Aspects to consider in the different test levels:

• **Project size**	• **Test Item Complexity**	• **Criticality of the test item**
• **Establishment and adherence to a calendar of activities**	• **Application area**	• **Budget for testing**
• **Availability and quality of documentation**	• **Testing environment**	• **Team skills**

Software Testing

Must be a process:

- Systematic
- Properly documented
- Reproducible and Repeatable
- Parallel to development



Otherwise, it is not possible to obtain **reliable results**.

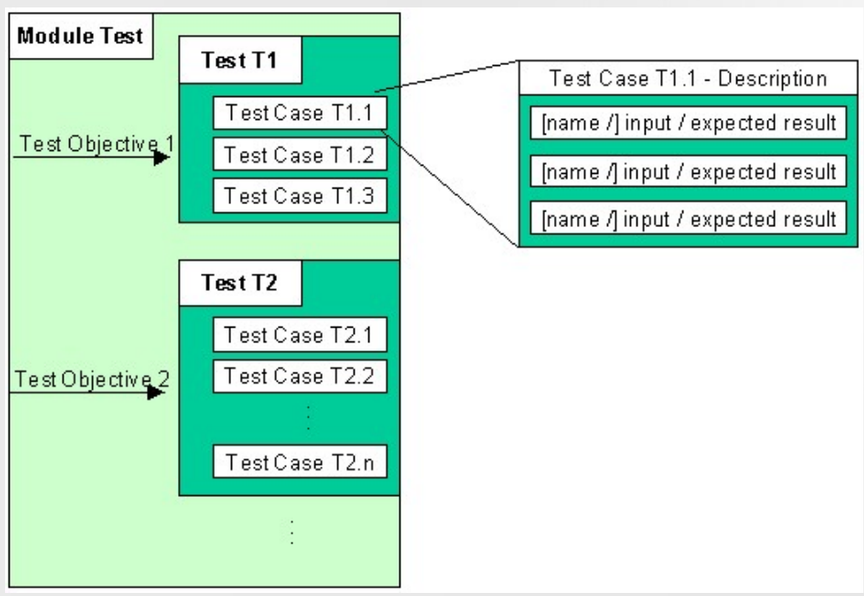
Software Testing – Module Testing

Goal	Action
• Execute Functionality as described. • Deterministic and reproducible behavior • Robustness against unexpected input values	• Define use cases in the appropriate document (Module Test Specification)
• Well-defined initialization • Correct configuration • Easy system integration	• Tests in a standard system, common to the entire team
• Absence of side effects	• Not possible to verify systematically, it is required to observe for a while whereas the system is tested

Software Testing – Module Testing

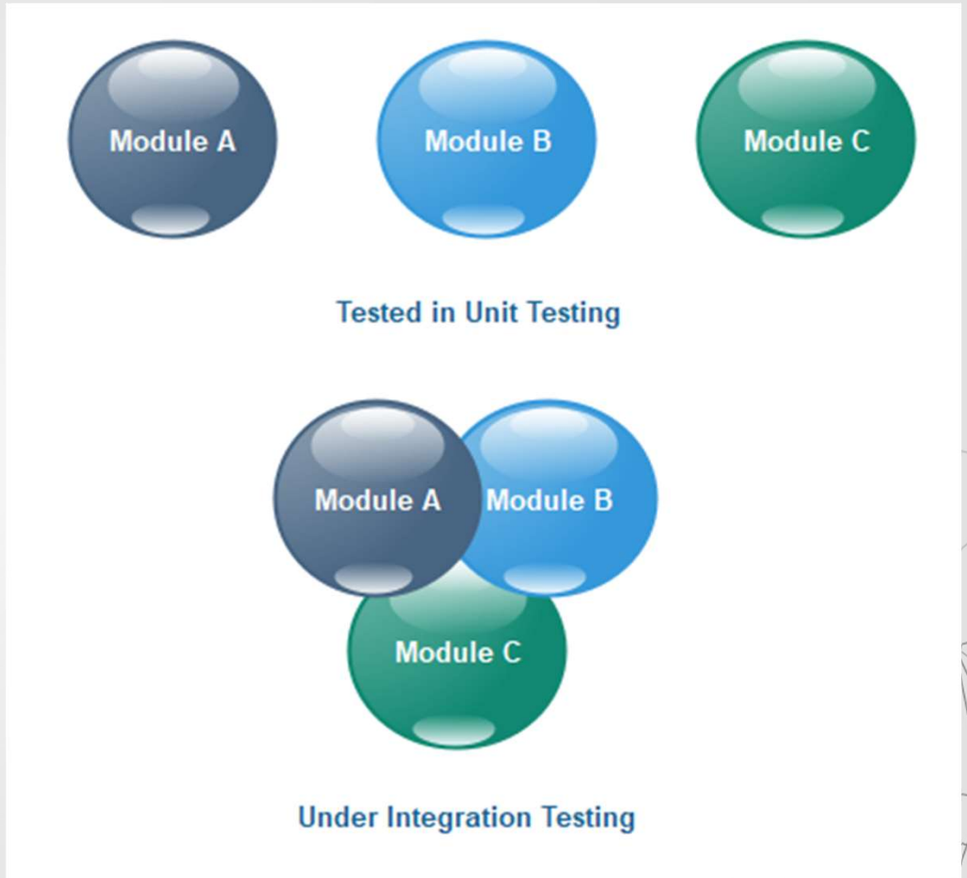
- Each Test Objective considers testing a certain functionality, it is made up of one or several Test Cases (test variants).
- A Test Case can be described as a series of inputs and their expected results

Example of a Module Test Specification (MTS) structure:



Software Testing – Integration Testing

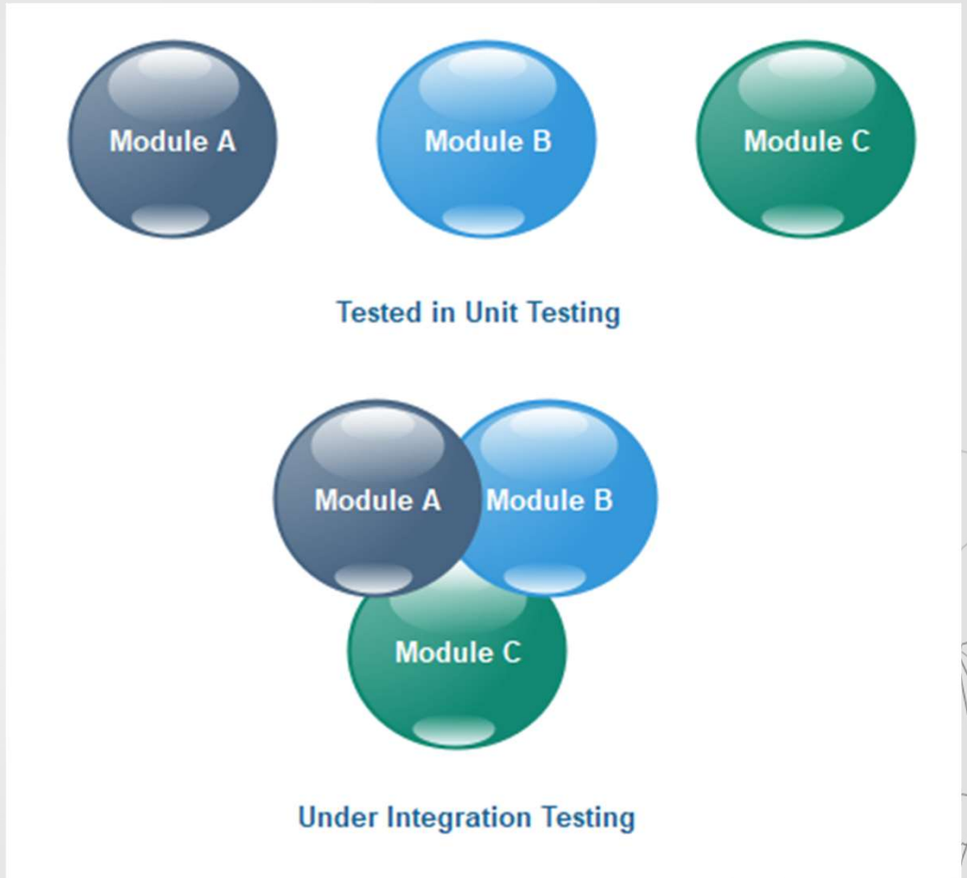
- Phase in software testing in which individual **software modules are combined and tested as a group.**
- Integration testing is conducted to **evaluate the compliance of a system** or component **with** specified **functional requirements.**
- Integration testing takes as its **input modules that have been unit tested**, groups them, applies tests defined in an Integration Test Plan, and **delivers as an integrated system** ready for system testing.



Software Testing – Integration Testing

Key reasons to conduct integration testing:

- Integrating disparate **modules into a working application**
- Ensuring that **changing requirements are incorporated** into the application
- Eliminating common issues missed during unit testing
- Eliminating other common problems

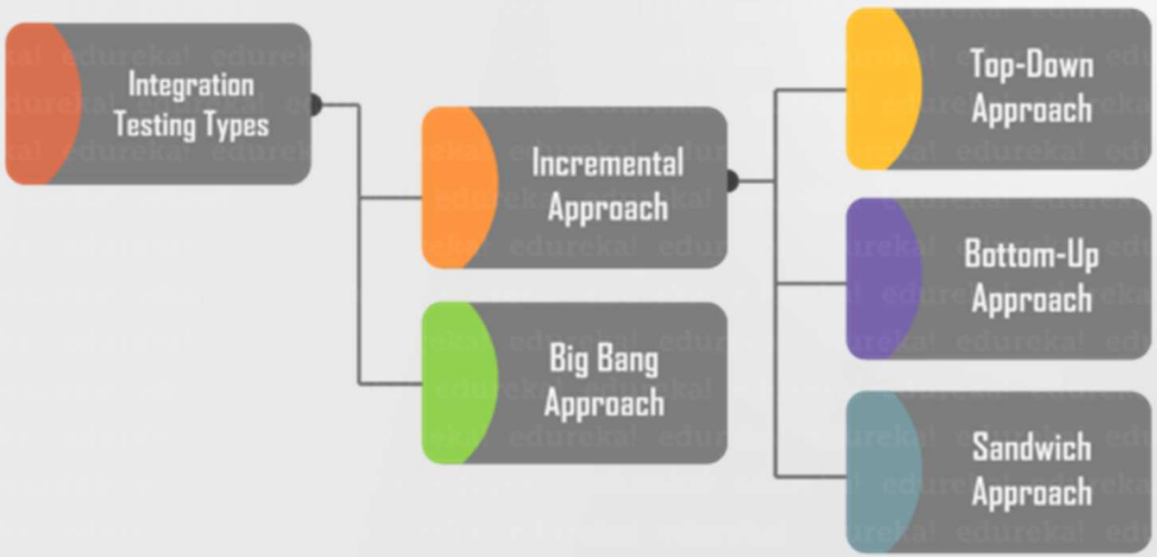


Software Testing – Integration Testing

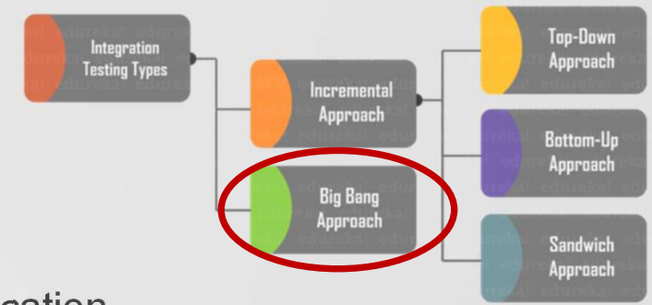
Common approaches to integration testing.

Four key strategies to execute integration testing derived from 2 types:

- **Big-bang**: involves integrating all modules at once and testing them all as one unit.
- **Top-down**: tests from the topmost or highest-level module and gradually proceeding to the lower modules.
- **Bottom-up**: tests lower-level modules first, and then gradually progressing incrementally to higher-level modules.
- **Sandwich/hybrid testing**: combines the top-down and bottom-up approaches.



Software Testing – Integration Testing



Big Bang testing approach

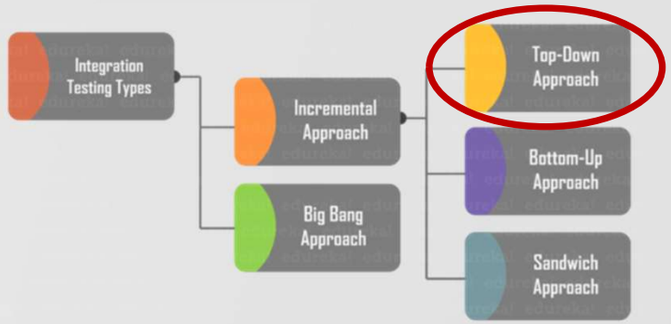
Advantages:

- Its **suitability** for testing **small systems**.
- Its **ease of identifying errors** in such systems, saving time and speeding up application deployment.

Disadvantages:

- **Locating the source of defects can be difficult** since different modules are integrated as one unit.
- Big-bang testing is time-consuming for a large system with numerous units.
- Testers **could miss** some interface links or **bugs**.
- Testers must **wait until all modules are available**, so they have less time to do the testing and developers have less time to fix any errors.
- Due to simultaneous testing, high-risk critical modules and peripheral modules dealing with user interfaces are **not tested on priority** (as they should be).

Software Testing – Integration Testing



Top-down testing approach

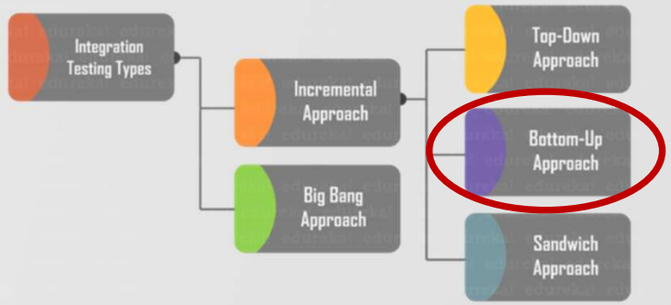
Advantages:

- It is **easier to identify defects** and isolate their sources.
- Testers check important units first, so they are more **likely to find critical design flaws**.
- It is possible to create an early prototype.

Disadvantages:

- The **examination of lower-level modules can take a lot of time**, so testers may not test them adequately or completely.
- When too many testing stubs are involved, the testing process can become complicated.

Software Testing – Integration Testing



Bottom-up testing approach

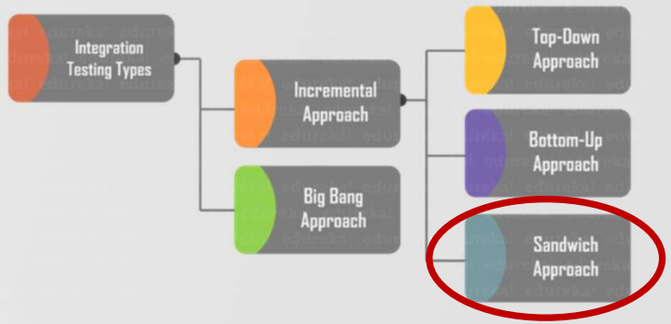
Advantages:

- It is **easier to find and localize faults**.
- **Less time** is needed **for troubleshooting** since testers don't have to wait for all modules to be available for testing.

Disadvantages:

- Testing all modules can take a lot of time, so there may be delays in releasing the final product.
- **Critical modules are tested only in the final stages**, so testers may miss some defects and developers may not have enough time to fix found defects.
- Testing can be **complicated if the software consists of multiple low-level units**.
- It is **not possible to create an early prototype**.

Software Testing – Integration Testing



Sandwich testing approach

Advantage:

- **Preferable for testing huge programs** like operating systems or other long-term applications.

Disadvantages:

- It is complex and costly.
- A **higher level of precision and skill are required** for this approach.

Software Testing – SW System Testing

System Testing is a black box category testing

- Testing **conducted on a complete integrated system** to evaluate the system's compliance with its specified requirements.
- The purpose of a system test is to **evaluate the end-to-end system specifications**. Usually, the software is only one element of a larger computer-based system.
- There are several types of System Testing, below we have listed types of system testing that most of software development companies use:
  - Load testing
  - Regression testing
  - Functional testing
  - Recovery testing



Software Testing – SW System Testing



- **Load testing** - is necessary to know that a software solution will perform under real-life loads.



- **Regression testing** - involves testing done **to make sure none of the changes made over the course of the development process have caused new bugs**. It also makes sure no old bugs appear from the addition of new software modules over time.



- **Functional testing** - Also known as **functional completeness testing**, Functional Testing involves trying to think of any possible missing functions. Testers might make a list of additional functionalities that a product could have to improve it during functional testing.



- **Recovery testing** - is done to demonstrate a **software solution is reliable**, trustworthy and can successfully recoup from possible crashes.

Acceptance Testing

- System **testing performed by the customer** himself:
  - To determine whether the system should be accepted or rejected.

- Execute the Customer Acceptance Test before delivery (if possible).

- Quality First



Conformance Testing



Test technique:	**Conformance Testing**
Description:	Conformance testing is testing to determine whether a system **meets some specified standard**. In this context we test, if the behaviour of the system conforms to its specification.
Goal:	Show that the behaviour of the system conforms to its specification
Example:	This type of testing is **used throughout the initial stages of development** to ensure the accuracy of a protocol implementation. It is also employed in regression testing after initial product deployment to double-check any changes in the implementation.

Questions?

